

P2516R0

string_view is implicitly convertible from what?

Published Proposal, 2021-12-31

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Introduction

2 Problems

2.1 Is **vector** a string?

2.2 Type unsafety

2.3 Nonexisting practice

3 Alternatives

4 Proposal

5 Acknowledgements

References

Informative References

§ 1. Introduction

As found from usage experience, implicit conversions introduced by [P1391] have undesirable side effects and break existing use cases for `string_view` as a string-reference type ([N3921]). Moreover, [P1391] uses contiguity as a proxy for detecting if a type is string-like which appears to be conceptually wrong. This paper proposes removing the problematic conversions.

§ 2. Problems

[P1391] made `string_view` and, more generally, `basic_string_view` implicitly constructible from any contiguous range of characters and, as it turned out but not mentioned in the paper, not just characters. Like `vector<bool>` this seemed like a good idea at the time even to the author of the current paper who didn't put too much thought into the implications or whether the design is particularly sound. Unfortunately even a brief exposure to the actual implementation and initial usage experience revealed severe issues, some of which are summarized in this paper.

§ 2.1. Is `vector` a string?

Consider the following simple example:

```
template <typename Container>
auto print(const Container& c)
    -> std::enable_if_t
```

The `print` function takes an argument and prints it either as a quoted string or as a comma-separated list of values delimited by `[]`.

What will `print(std::vector{'a', 'b'})` output?

Thanks to newly added implicit conversions the answer depends on the C++ standard version. In C++17 - C++20 this prints `[a, b]` as expected while in the upcoming C++23 the output suddenly changes to `"ab"`.

In C++17 - C++20 it was perfectly reasonable to assume that `string_view` means a reference to something string-like such as `std::string` or a string literal. Quoting [N3921] that introduced `string_view`:

Google, LLVM, and Bloomberg have independently implemented a string-reference type to encapsulate this kind of argument. `string_view` is implicitly constructible from `const char*` and `std::string`.

The string nature of `string_view` is also indicated in its name, its API and the fact that `basic_string_view` takes character traits as a template parameter.

The new implicit conversions broke that assumption, effectively changing the meaning of `string_view` to denote not a reference to a string-like type but a reference to an arbitrary contiguous range.

Conceptually the problem with these conversions is that they confuse representation with semantics using contiguity as a proxy for being a string. Consider

```
print(std::list{'a', 'b'});  
print(std::deque{'a', 'b'});  
print(std::vector{'a', 'b'});
```

	C++17	C++20	C++23
Output	<code>[a, b]</code>	<code>[a, b]</code>	<code>[a, b]</code>
	<code>[a, b]</code>	<code>[a, b]</code>	<code>[a, b]</code>
	<code>[a, b]</code>	<code>[a, b]</code>	<code>"ab"</code>

Why is `vector` different from other containers and is it really "string-like" as implicit convertibility to `string_view` suggests?

Of course we could change the definition of `print` to workaround the issue but this won't fix the underlying problem. We no longer have a string-reference type which invalidates the goal of [N3921]. Instead we have a contiguous-range-reference type with a misleading name and a string-like API. The introduction of `std::span` in C++20 makes this design look even stranger because `std::span<const T>` is a natural representation for the above type.

If this wasn't bad enough, the same applies to non-character types as well, enabling such fun examples as:

```
std::basic_string_view s = std::vector{42.0};
```

or, more practically, a generic version of the `print` example above where a contiguous range of non-characters can be printed as a pseudo-string.

This is not a theoretical problem. There were at least two bug reports in `{fmt}`, an open-source formatting library ([[FMT-BUG-2585](#)], [[FMT-BUG-2634](#)]), of subtle breakages related to this change even though it requires opting into an experimental C++23 standard library implementation which is very uncommon.

To solve the problem in `{fmt}` we'll have to indefinitely continue using a replacement for `std::string_view` together with a few workarounds with no chance of eventually converging on `std::string_view` as a string-reference type. The same is likely true for some other text processing and serialization use cases that need to distinguish between strings and containers.

§ 2.2. Type unsafety

Another problem is that `char` has double meaning and is used as a code unit type or as a byte depending on the context. Additional semantic context may be added by types built on top of `char`, some of which may now become unexpectedly convertible to `string_view`. As pointed out by users, `vector<char>` and `span<char>` are commonly used as byte buffers and implicit conversions could introduce type safety problems.

§ 2.3. Nonexisting practice

Maybe [[P1391](#)] standardizes existing practice? Let's look at the types that inspired `string_view`:

- LLVM's `StringRef`
- Google's `StringPiece`

None of them provides a constructor from a contiguous range or even a vector. So this feature doesn't standardize existing practice but is completely novel which explains why even a brief exposure to the implementation revealed a number of issues.

§ 3. Alternatives

The main part of the motivation of [P1391] is compelling:

While P1206 gives a general motivation for range constructors, it's especially important for `string_view` because there exist in a lot of codebases string types that would benefit from being convertible to `string_view`. For example, `llvm::StringRef`, `QByteArray`, `fbstring`, `boost::container::string` ...

However, the solution is overreaching and as shown above breaks the main use case for a string-like reference type by introducing semantically lossy implicit conversions.

Whether a type is string-like should generally be controlled by the class author, not detected via some heuristic. We already have a mechanism for this that is used in `std::string`, namely `operator string_view`. If it is insufficient a proper solution would be to introduce another opt-in mechanism such as a trait that specifies if the type is string-like and is eligible for conversion into a `string_view`. The latter is not proposed by the current paper which only tries to mitigate the damage done by [P1391] before it is too late.

§ 4. Proposal

Remove wording introduced by [P1391] from the standard.

§ 5. Acknowledgements

Thanks Matthias Moulin and Barry Revzin for independently bringing up this issue.

§ References

§ Informative References

[FMT-BUG-2585]

Matthias Moulin. [Some ranges of char are misprinted or don't compile](https://github.com/fmtlib/fmt/issues/2585). URL: <https://github.com/fmtlib/fmt/issues/2585>

[FMT-BUG-2634]

Barry Revzin. [Some ranges of char are misprinted or don't compile](https://github.com/fmtlib/fmt/issues/2634). URL: <https://github.com/fmtlib/fmt/issues/2634>

[N3921]

Jeffrey Yasskin. [string_view: a non-owning reference to a string](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3921.html). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3921.html>

[P1391]

Corentin Jabot. [Range constructor for std::string_view](https://wg21.link/p1391). URL: <https://wg21.link/p1391>